● 問題 1

【結果】

【マニュアル】

① 計算結果出力ラベル

入力された値を出力する。

② 2 進数表記出力ラベル

① の 2 進数表記を常に表示する

③ 進数機能

10 進数、8 進数、16 進数を選択でき、選んだ進数表記に上のラベルの数を変更する。また、選んだ進数によってボタンの無効化なども行われる。

④ 符号機能

符号なし、符号ありが選択でき、符号アリになった場合ラベルの数がマイナス表記に変更される。

⑤インジケータボタン:A～F は 16 進数用のボタン、9,8 は 16 進数用、10 進数のボタン、それ以外は 10、16、8 進数全てに対応したボタンであり、進数機能で選んだ進数に対応したボタンが使用可能である。(8 進数の場合、対応していない A などのボタンは無効化されている。)

⑥ FF 機能

FF は 16 進数の F を 2 回入力するボタンである。16 進数を選んでいる時のみ有

効化される。

⑦00 機能
0 を 2 回入力するためのボタンである。

⑧ Clear 機能
基数計算機の状態を初期化し、再度 0 からの計算を可能にするボタン。四則演算は無効化される。

⑨ 履歴機能
現在のフォアグラウンドな基数計算するためのパネルから移動し、履歴を表示するパネルに移行する。ここでは、計算機の数字が変更されるたびに入力されている数字が記録され、どのような計算がされてきたのかという履歴が表示される。

⑩ 足し算機能
現在入力されている値に対して新しく入力した値を足し合わせる機能
数字が入力されていない間はボタンが無効化される
計算結果出力機能を使わず、2 回目以降に再度押した場合、1 つ前の計算結果に対しての演算が行われる。

⑪ 引き算機能
現在入力されている値に対して新しく入力した値を引いていく機能
数字が入力されていない間はボタンが無効化される
計算結果出力機能を使わず、2 回目以降に再度押した場合、1 つ前の計算結果に対しての演算が行われる。

⑫ 計算結果出力機能(=)
四則演算した結果を求める機能
足し算機能等で足し合わせた場合または引き算機能で引いた結果を反映する際に使用する。
足し算機能または引き算機能が使われていない場合はボタンが無効化される

⑬戻る機能
インジケータボタンが押される以前の状態まで戻ることができる機能
最大まで戻った時に戻るボタンは無効化される

⑭進む機能
戻る機能によって戻った際に有効化され、戻る前の状態に戻す事ができる機能
最大まで進んだ際に進むボタンは無効化される

【ソースコード】
・ main メソッド

```
public static void main(String[] args){
    JFrame f = new JFrame("基数計算機");
    RadixConv2 rc = new RadixConv2();

    f.add(rc);
    f.pack();
    f.setVisible(true);
}
```

JFrame、RadixConv2 オブジェクトを生成し、f に rc を追加し、rc のサイズに合わせてから表示している。

コンストラクタ

```java
RadixConv2(){
    JPanel main, bmain, sp, cpp;
    String bcbname[] = {"10進数", "2進数","8進数", "16進数"};
    String hcbname[] = {"符号なし", "符号あり"};

    main = new JPanel(); bmain = new JPanel(); sp = new JPanel(); cpp = new JPanel();
    JComboBox bcb = new JComboBox(bcbname), hcb = new JComboBox(hcbname);
    calc.setFont(new Font("", Font.PLAIN, 40));

    setLayout(new BorderLayout()); cpp.setLayout(new GridLayout(1, 0));
    main.setLayout(new BoxLayout(main, BoxLayout.Y_AXIS)); bmain.setLayout(new GridLayout(0, 3));
    sp.setLayout(new BoxLayout(sp, BoxLayout.Y_AXIS));

    String bname[] = {
            "D", "E", "F",
            "A", "B", "C",
            "7", "8", "9",
            "4", "5", "6",
            "1", "2", "3",
            "FF", "0", "00"
    };
    String spname[] = {"Clear", "履歴", "取り消し"};
    bb = new JButton[bname.length];
    spb = new JButton[spname.length];
    for(int i = 0; i < bb.length; i++){
        bb[i] = new JButton(bname[i]);
        bb[i].addActionListener(this); bb[i].setActionCommand(bname[i]);
        bmain.add(bb[i]);
    }
    for(int i = 0; i < spb.length; i++){
        spb[i] = new JButton(spname[i]); sp.add(spb[i]);
        spb[i].setMaximumSize(new Dimension(100, spb[i].getMaximumSize().height)); //サイズ統一
    }

    bcb.addActionListener(this); bcb.setActionCommand("bcb");

    cpp.add(bcb); cpp.add(hcb); main.add(cpp); main.add(bmain);
    add(calc, BorderLayout.NORTH); add(main, BorderLayout.WEST); add(sp, BorderLayout.EAST);
}
```

パネルに乗せるコンポーネントの実装を行っている。大まかな構成として、入力された値を出力する calc ボタン、進数、符号、インジケータボタンを配置する main パネル、特殊ボタンを表示する sp パネルで構成される。レイアウトとして、BorderLayout をとり、calc を North、main を West、sp を East に配置している。main 及び sp にはそれぞれボタンが配置され、ボタンのイベント登録やアクションコマンドの設定などが行われている。

actionPerformed メソッド

```java
@Override
public void actionPerformed(ActionEvent e) {
    // TODO Auto-generated method stub
    String cmd = e.getActionCommand();
    if(cmd.equals("hoge")){

    } else {
        register = register * 10 + Integer.parseInt(cmd);
        calc.setText(register + "");
    }
}
```

インジケータボタンは複数の Button オブジェクトで生成されているため。秘湯1 つの条件を書いてしまうとソースコードが膨大になってしまうことから、送られてきたイベントから取得できるコマンドが hoge(未設定)以外の場合にインジケータボタンの処理にうつる。
インジケータボタンのコマンドにはそれぞれの数字が設定されていることから、コマンドがインジケータボタンの数字となる。押したボタンの数字をそれまで表示されていた数字の後ろに追加したいことから、現在表示されている数字の10 進表記の値を格納している register を 10 倍し、それに cmd を足すことでそれまでの数字の後ろにインジケータの数字を追加することができる。求めた値を register に追加した後、register の値を 10 進数表記のまま数値を表示するラベルに設定することで実現している。

【ソースコード追加部】

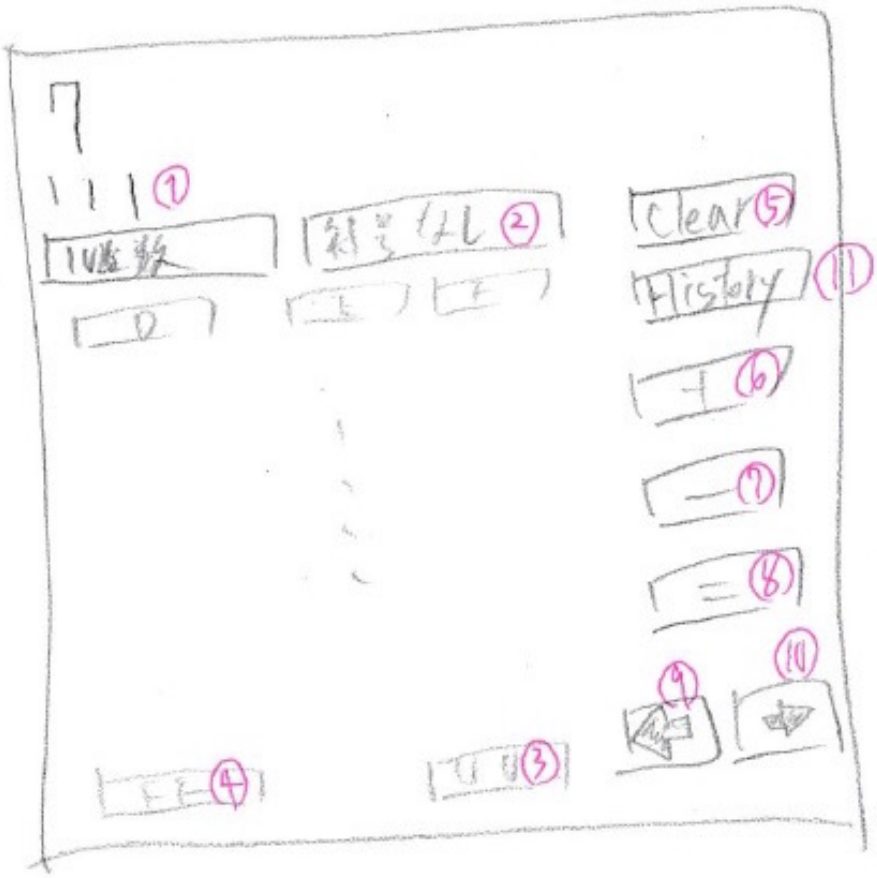
```java
@Override
public void actionPerformed(ActionEvent e) {
    // TODO Auto-generated method stub
    String cmd = e.getActionCommand();
    int base[] = {10, 8, 16};
    if(cmd.equals("bcb")){
        selectBase = ((JComboBox)e.getSource()).getSelectedIndex();
    }else {
        try{
            register = register * base[selectBase] + Integer.parseInt(cmd);
        } catch(NumberFormatException er){
            register = register * base[selectBase] + Integer.parseInt(cmd, 16);
        }
    }
    outputCalc();
}
```

基数コンボボックスには bcb というアクションコマンドが設定されている。イ

ベントから取得したコマンドが bcb の時に現在セレクトされたインデックスを selectBace というインスタンス変数に格納している。bcb 以外の時(インジケータボタンをおした時)、selcetBace で示している数を基数として register を計算している。結果を出力するラベルに register を設定する。
そして最後に outputCalc()によって数を示すラベルに基数コンボボックスで選択している進数表記で register を設定する。

```
private void outputCalc(){
    switch(selectBase){
    case 0:
        calc.setText(register + "");
        break;
    case 1:
        calc.setText("0" + Integer.toOctalString(register));
        break;
    case 2:
        calc.setText("0x" + Integer.toHexString(register));
        break;
    }
}
```

コンボボックスで選択されているインデックスを示す selectBace を評価し、0(10 進数)の場合、register をそのまま clac に設定している。1(8 進数)の場合、register を 8 進数表記に変換し calc に設定している。2(16 進数)の場合、register を 16 進数表記に変換し calc に設定している。


1
2
Clear
5
History
11
6
7
8
9
10
3
4

番号	機能	UI	機能説明
①	2進表示機能	ラベルによる出力	常時表示されている結果を2進数として表示する
②	符号機能	コンボボックス	符号のありなしを決める機能 有りの場合値がマイナス値となる
③	00機能	ボタン	一度に0を2つ入力できるボタン
④	FF機能	ボタン	1度にFを2つ入力できるボタン 進数機能によって16進が選択されている場合のみボタンが有効化される。
⑤	Clear機能	ボタン	基数計算機の状態を初期化し、再度0からの計算を可能にするボタン
⑥	足し算機能	ボタン	現在入力されている値に対して新しく入力した値を足し合わせる機能 数字が入力されていない間はボタンが無効化される 計算結果出力機能を使わず、2回目以降に再度押した場合、1つ前の計算結果に対しての演算が行われる。
⑦	引き算機能	ボタン	現在入力されている値に対して新しく入力した値を引いていく機能 数字が入力されていない間はボタンが無効化される 計算結果出力機能を使わず、2回目以降に再度押した場合、1つ前の計算結果に対しての演算が行われる。
⑧	計算結果出力機能(=)	ボタン	四則演算した結果を求める機能 足し算機能等で足し合わせた場合または引き算機能で引いた結果を反映する際に使用する。 足し算機能または引き算機能が使われていない場合はボタンが無効化される
⑨	戻る機能	ボタン	インジケータボタンが押される以前の状態まで戻ることができる機能 最大まで戻った時に戻るボタンは無効化される
⑩	進む機能	ボタン	戻る機能によって戻った際に有効化され、戻る前の状態に戻す事ができる機能 最大まで進んだ際に進むボタンは無効化される
⑪	History機能	ボタン、パネルの移動	それまでに計算された値が記録され、その履歴を見ることができる機能
	ボタン無効機能	ボタン	それぞれのボタンに対し、そのボタンが対応する状態の時に有効化され、それ以外の場合は無効化される機能

【構成】
RadixConv4 クラス(main)
ChangeFrame クラス:フォアグラウンドなパネルを切り替えることができるフレームクラス。
History パネル;履歴を表示させるパネルクラス。メインのパネルに戻るボタンがある。

【ソースコード】
・ ChangeFrame クラス

```java
public class ChangeFrame extends JFrame{
    ArrayList<Component> ChangeFrameElements = new ArrayList<Component>

    ChangeFrame(){}
    ChangeFrame(String title){super(title);}

    @Override
    public Component add(Component comp){
        super.add(comp);
        ChangeFrameElements.add(comp);
        return comp;
    }
    public void changePanel(int idx){
        for(int i = 0; i < ChangeFrameElements.size(); i++){
            ChangeFrameElements.get(i).setVisible(false);
        }
        ChangeFrameElements.get(idx).setVisible(true);
    }
}
```

JFrame を継承しているフレームクラスである。ChangeFrame オブジェクトに add した際に ChangeFrameElements という ArrayList にコンポーネントが格納され、追加したコンポーネントが保持される。changePanel メソッドにより渡された引数を arrayList のインデックスとして、保持しているコンポーネントの中で指定されたコンポーネントだけを表示するようにしている。これにより、フォラグラウンドなパネルの変更を可能にしている。

・ History クラス

```
public class HistoryPanel extends JPanel implements ActionListener{

    ChangeFrame cf;

    HistoryPanel(){
        JButton mBack = new JButton("Back");
        mBack.addActionListener(this); mBack.setActionCommand("mainBack
        add(mBack);
        setLayout(new BoxLayout(this, BoxLayout.Y_AXIS));
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        // TODO Auto-generated method stub
        cf.changePanel(0);
    }
}
```

JPanel を継承したパネルクラスである。また、ActionListener を実装することでイベント処理を可能にしている。コンストラクタでは計算機パネルに戻るための Back ボタンが生成されている。
Back ボタンが押された際に ChangeFrame の changePanel メソッドを行うことで計算機パネルに戻ることができる。

・RadixConv4 クラス
コンストラクタ(差分)

```
ImageIcon back = new ImageIcon(getClass().getClassLoader().getResource("sd114/back.png"
        next = new ImageIcon(getClass().getClassLoader().getResource("sd114/next.png"))
actionHistory.add(0);
String spname[] = {"back", "next", "Clear", "History", "+", "-", "="};
bb = new JButton[bname.length]; spb = new JButton[spname.length];
for(int i = 0; i < bb.length; i++){
    bb[i] = new JButton(bname[i]);
    bb[i].addActionListener(this); bb[i].setActionCommand(bname[i]);
    bmain.add(bb[i]);
}
for(int i = 0; i < 6; i++){bb[i].setEnabled(false);}
bb[15].setEnabled(false);

for(int i = 2; i < spb.length; i++){
    spb[i] = new JButton(spname[i]); sp.add(spb[i]);
    spb[i].addActionListener(this); spb[i].setActionCommand(spname[i]);
    spb[i].setMaximumSize(new Dimension(100, spb[i].getMaximumSize().height)); //サイズ
}
for(int i = 4; i < 7; i++){spb[i].setEnabled(false);}
spb[0] = new JButton(back); spb[1] = new JButton(next);
spb[0].addActionListener(this); spb[0].setActionCommand(spname[0]); spb[1].addActionLis

bnp.setMinimumSize(new Dimension(0, 0));
spb[0].setEnabled(false); spb[1].setEnabled(false);
bcb.addActionListener(this); bcb.setActionCommand("bcb"); hcb.addActionListener(this);
sp.setPreferredSize(new Dimension(100, sp.getMaximumSize().height));
bnp.add(spb[0]); bnp.add(spb[1]);

sp.add(bnp); cpp.add(bcb); cpp.add(hcb); calcp.add(calc); calcp.add(twoCalc);
main.add(cpp); main.add(bmain);
add(calcp, BorderLayout.NORTH);
add(main, BorderLayout.WEST); add(sp, BorderLayout.EAST);
```

戻る、進むボタン用にイメージアイコンを生成している。
actionHistory は入力された値を格納しておく arraylist であり、戻る進む機能の実装に使用する。最初の文字が 0 であることから 0 を追加している。
各種ボタンを設定し、最初は 10 進数であることから A～F、FF、数字を入力していないことから、足し算、引き算、イコールボタン、戻る進むボタンを無効化している。
最後にレイアウトを整え追加している。

・ main スレッド

```
public static void main(String[] args){
    new RadixConv4().doIt();
}

private void doIt(){
    ChangeFrame f = new ChangeFrame("基数計算機");
    RadixConv4 main = new RadixConv4();
    HistoryPanel history = new HistoryPanel();
    JScrollPane jsp = new JScrollPane(history);
    f.setResizable(false);
    history.cf = f;
    main.cf = f; main.history = history;

    f.add(main); f.pack();
    f.add(jsp);

    f.changePanel(0);
    f.setVisible(true);
}
```

切り替えフレームである ChangeFrame、自身、履歴パネルである history、スクロールパネルである JScrollPane(history 設定)のオブジェクトを生成している。
ChangeFrame のサイズを変更できないようにしている。
それぞれインスタンス変数にオブジェクトの追加などを行い、ChangeFrame オブジェクトに追加された main パネルを表示している。

・ actionPerformed メソッド

```
@Override
public void actionPerformed(ActionEvent e) {
    // TODO Auto-generated method stub
    String cmd = e.getActionCommand();

    if(cmd.equals("Clear")){
        this.historyIdx = 0; this.register = 0; this.registerTmp = 0;
        actionHistory.clear(); enzanJudge = -1;
        calc.setText("0"); actionHistory.add(0); twoCalc.setText("0");
        for(int i = 4; i < 7; i++){spb[i].setEnabled(false);}
        spb[0].setEnabled(false); spb[1].setEnabled(false);
```

初期化ボタンが押された際の処理である。各種データの初期化、ボタンの無効化を行っている。

```
else if(cmd.equals("History")){
  cf.changePanel(1);
```

History ボタンが押された時の処理である。履歴パネルをフォアグラウンドにしている。

```
} else if(cmd.equals("back")){
    historyIdx--; //System.out.println(historyIdx +";"+ actionHistory.
    register = actionHistory.get(historyIdx);
    outputCalc();
    if(historyIdx == 0){spb[0].setEnabled(false);}
    spb[1].setEnabled(true);
} else if(cmd.equals("next")){
    register = actionHistory.get(++historyIdx);
    outputCalc();
    if(historyIdx == actionHistory.size() - 1){ spb[1].setEnabled(fals
    spb[0].setEnabled(true);
```

戻るボタン、進むボタンが押された時の処理である。
historyIdx は現在どの位置の履歴を見ているのかを表すインスタンス変数である。
戻るの場合、historyIdx を 1 つ減らしそれより一つ前の履歴を register に追加し outputCalc メソッドで計算結果を出力している。historyIdx が 0 である(これ以上遡れる履歴がない場合)戻るボタンを無効化している。そして戻った場合それ以降の履歴があることから進むボタンを有効化している。これと真逆の動作が進むボタンの処理に書かれている。

```
} else if(cmd.equals("+") || cmd.equals("-")){
    for(int i = 4; i < 7; i++){spb[i].setEnabled(true);}
    if(enzanJudge == -1) {
        registerTmp = register;
    } else if(enzanJudge == 0){
        registerTmp += register;
    } else if(enzanJudge == 1){
        registerTmp -= register;
    }

    if(cmd.equals("+")){
        enzanJudge = 0;
    } else {
        enzanJudge = 1;
    }
    calc.setText(registerTmp + "");
    twoCalc.setText(Integer.toBinaryString(registerTmp));
    history.add(new JLabel(registerTmp + ""), 1); history.add(new JLabel(cmd + reg
    spb[6].setEnabled(true);
    register = 0;
```

+ボタン、-ボタンが押された時の処理である。
+,-,=ボタンを有効化している。
enzanJudge というインスタンス変数は-1 の時それ以前に+及び-が押されていないことを示し、0 の時は前回+、1 の時は前回-ボタンが押されていることを示す。-1 の時は基準である値を registeTmp に代入し、0 の場合はそれ以前の値である registerTmp に現在入力された値を足しあわせた値を registerTmp に代入し、1 の場合はそれ以前の値に現在の値を引いた値を registerTmp に追加している。
次に+ボタンが押された際に enzanJudge を 0 にしてプラスが押されたことを明示し、それ以外の場合はマイナスが押されたことを示す 1 を追加している。
次に計算された registerTmp をラベルに設定することで四則演算がされる。
history は ChangeFrame に追加されている history パネルであり、これに registerTmp の値をラベルとして追加することで履歴を作ることができる。
そして、再度入力できるように register を 0 にしている。

```
else if(cmd.equals("=")){
  if(enzanJudge == 0){
      register += registerTmp;
  } else {
      register = registerTmp - register;
  }
  enzanJudge = -1;
  outputCalc(); registerTmp = 0;
  history.add(new JLabel("=" + calc.getText()), 1);
else {
```

四則演算の結果を反映させるイコールボタンの処理である。
=の前にプラスが押されていた場合、現在の値にそれまでの値を追加したものを register に再度代入している。マイナスの場合はそれまでの値から現在の値を引いた値を register に追加している。
enzanJuge を-1 にすることで四則演算が完了したことを示すことができる。
outCalc()によって出力し、それまで計算した値を保持していた registerTmp を 0 に戻す。最後に現在の値を履歴パネルにラベルとして追加している。

```
} else {
    String outputName[] = {"進数変換:", "符号変換", "ボタン入力"};
    int idx;
    if(cmd.equals("bcb")) {
        selectBase = ((JComboBox)e.getSource()).getSelectedIndex();
        buttonEnabled(); idx = 0;
    } else if(cmd.equals("hcb")){
        selecthugo = ((JComboBox)e.getSource()).getSelectedIndex(); idx = 1;
    }else {
        calcAction(cmd);

        for(int i = historyIdx + 1; i < actionHistory.size(); i++) actionHistory
        actionHistory.add(++historyIdx, register); //0から
        if(historyIdx == actionHistory.size() - 1){ spb[1].setEnabled(false);}
        spb[0].setEnabled(true);
        spb[4].setEnabled(true); spb[5].setEnabled(true); idx = 2;
    }
    outputCalc();
    history.add(new JLabel(outputName[idx] + calc.getText()), 1);
}
```

上記以外のボタンの処理である。
さらにこの中で処理が 3 つにわかれている。
bcb は進数コンボボックスが変更された時の処理である。選択されたインデック

スを取得し、進数コンボボックスで選択されたインデックスを格納する selectBase に代入している。buttonEnabled メソッドによって選択された進数に応じてボタンの無効化などが行われる。
hcb は符号コンボボックスが変更された時の処理である。選択されたインデックスを取得し、符号コンボボックスで選択されたインデックスを格納する selecthugo に代入している。
それ以外の場合はインジケータボタンが押された時に呼び出される。calcAction()メソッドによって入力された値を使って register が再計算される。次に履歴 list から、現在の履歴インデックスの次の要素から保存されている最後の要素を全て削除し、現在の履歴インデックスの次の場所に現在の register の値を追加している。これにより戻るボタンなどで戻った後進むボタンが押せるようになった状態でインジケータボタンを押した際に、次の履歴が全て削除され、現在の履歴に上書きすることができる。
そして、現在の履歴が最後だった場合に進むボタンを無効化する。戻る、四則演算ボタンを有効化する。

最後に変更した register を outputCalc メソッドにより出力し、その値を履歴に追加する。

・ buttonAction メソッド

```
private void calcAction(String cmd){
    int base[] = {10, 8, 16};
        if(cmd.equals("00") || cmd.equals("FF")){
            register = register * base[selectBase] * base[selectBase] + Integer.parse
        } else {
            register = register * base[selectBase] + Integer.parseInt(cmd, 16);
        }
}
```

00 および FF が押された時は現在の値に基数を 2 回かけ、00 及び FF を足している。cmd を最後に足しているため、16 進数用のボタンを足した際に数字+文字になりエラーが出てしまうことから、cmd が 16 進数であることを明示することで回避している。

・ButtonEnable メソッド

```
private void buttonEnabled(){
    switch(selectBase){
    case 0:
        for(int i = 0; i < bb.length; i++){
            if(i < 6 || i == 15){
                bb[i].setEnabled(false);
            }else {
                bb[i].setEnabled(true);
            }
        }
        break;
    case 1:
        for(int i = 0; i < bb.length; i++){
            if((i < 15 && 8 < i) || 15 < i || i == 6){
                bb[i].setEnabled(true);
            } else {
                bb[i].setEnabled(false);
            }
        }
        break;
    case 2:
        for(int i = 0; i < bb.length; i++){
            bb[i].setEnabled(true);
        }
        break;
    }
}
```

進数コンボボックス選択されている進数表記で使えるボタンを有効化し、それ以外を無効化している。

・ outputCalc メソッド

```
private void outputCalc(){
    int tmp;
    if(selecthugo == 0){
        tmp = register;
    } else {
        tmp = -register;
    }
    switch(selectBase){
    case 0:
        calc.setText(tmp + "");
        break;
    case 1:
        calc.setText("0" + Integer.toOctalString(tmp));
        break;
    case 2:
        calc.setText("0x" + Integer.toHexString(tmp));
        break;
    }
    twoCalc.setText(Integer.toBinaryString(tmp));
}
```

符号コンボボックスで符号無しが選ばれている時にプラス、それ以外はマイナス表記に変換して tmp に代入する。
tmp を進数に応じて結果を示すラベルに設定している。最後に 2 進数表記を表示する twoCalc ラベルに 2 新表記の tmp を設定している。